Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 7

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html

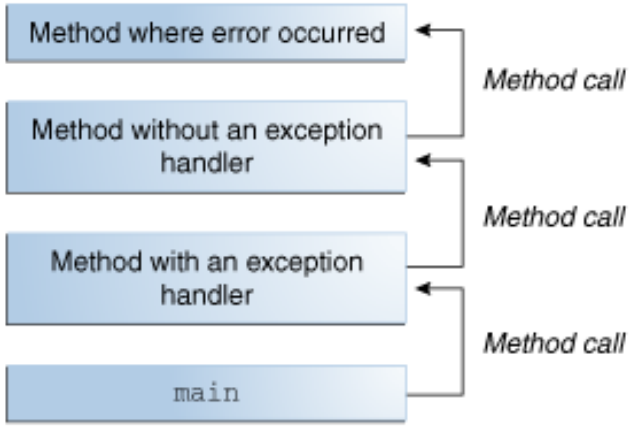
Trail: Essential Classes> Exceptions

The Java programming language uses *exceptions* to handle errors and other exceptional events.

- What Is an Exception?
- The Catch or Specify Requirement
- How to Throw Exceptions
- The try-with-resources Statement
- Unchecked Exceptions — The Controversy
- Advantages of Exceptions

What Is an Exception?

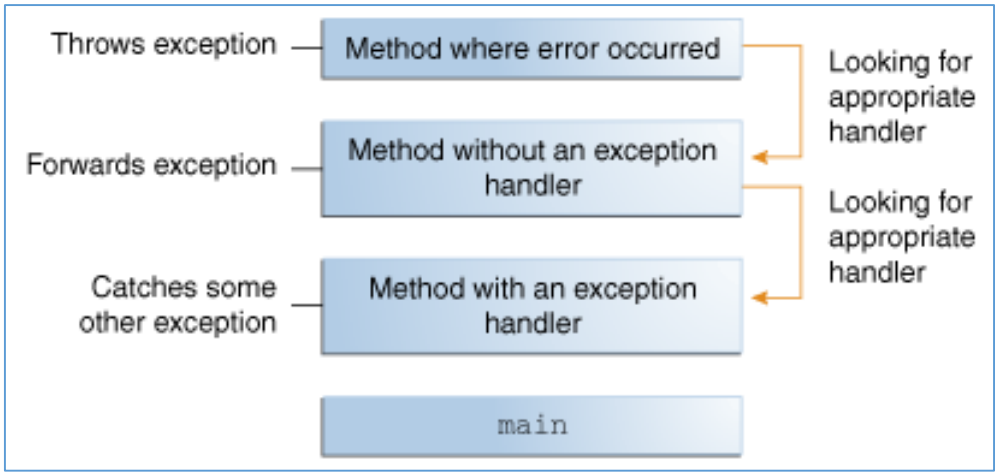
- The term ***exception*** is shorthand for the phrase "exceptional event."
- An **exception**
  - is an *event*, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
  - We use exceptions to manage errors
- When an **error** occurs within a **method**,
  - the method creates an object and hands it off to the *runtime system*.
  - The object, called an **exception object**, contains information about the error, including its type and the state of the program when the error occurred.
  - Creating an exception object and handing it to the runtime system
    - is called throwing an exception.
- After a method **throws** an ***exception***,
  - runtime system attempts to find something to **handle** it.
  - The set of possible "somethings" to handle the exception is the ordered list of methods that had been called to get to the method where the error occurred. T
  - The list of methods is known as the **call stack**.



What Is an Exception?

- After a method **throws** an ***exception***
  - The runtime system searches the call stack for a method that contains a block of code that can handle the exception.
  - This block of code is called an ***exception handler***.
  - The search begins with the method in which the error occurred and proceeds through the **call stack** in the reverse order in which the methods were called.
  - When an appropriate handler is found, the runtime system passes the exception to the handler.
    - The exception handler is said to catch the exception
  - An **exception handler** is considered appropriate if the type of the exception object thrown matches the type that can be handled by the handler.

- If the runtime system searches all the methods on the call stack
  - without finding an appropriate exception handler,
  - the program terminates.



The Catch or Specify Requirement

- **The Catch or Specify Requirement**
  - A code that might throw certain **exceptions** must be enclosed by either of the following:
  - A **try** statement that catches the exception.
    - The try must provide a handler for the exception.
  - A **method** that specifies that it can throw the exception.
    - The method must provide a throws clause that lists the exception
- **Not all** exceptions are subject to the Catch or Specify Requirement.

- Example: Handling a file
  - If user provides non-existed File and you pass it to construction of the *FileReader* object, the constructor throws ***java.io.FileNotFoundException***.
    - Your program should catch this exception and notify the user of the mistake, asking for a corrected file name
  - Suppose program is unable to read the file because of a hardware malfunction. The unsuccessful read will throw ***java.io.IOError.***
    - You may want the program to print a stack trace and exit.
  - If a logic error causes a null to be passed to the constructor of the *FileReader* object, the constructor will throw ***NullPointerException***.
    - You want to eliminate the bug that caused the exception to occur.

The Catch or Specify Requirement

- **The Three Kinds of Exceptions**
  - ***Checked exception.***
    - Exceptional conditions that an application should anticipate and recover from.
    - Checked exceptions are subject to the **Catch or Specify Requirement**.
    - All exceptions are checked exceptions, except for those indicated by *Error*, *RuntimeException*, and their subclasses.
  - ***Error***
    - Exceptional conditions that are external to the application, and the application usually cannot anticipate or recover from.
    - Errors are not subject to the Catch or Specify Requirement.
    - Errors are those exceptions indicated by *Error* and its subclasses.
  - ***Runtime exception***
    - Exceptional conditions that are internal to the application, and that the application usually cannot anticipate or recover from.
    - These usually indicate programming bugs, such as logic errors or improper use of an API.
    - Runtime exceptions are not subject to the Catch or Specify Requirement.
    - Runtime exceptions are those indicated by *RuntimeException* and its subclasses.
  - Errors and runtime exceptions are known as **unchecked exceptions**.

```java
// Note: This class will not compile yet.
import java.io.*;
import java.util.List;
import java.util.ArrayList;

public class ListOfNumbers {

    private List<Integer> list;
    private static final int SIZE = 10;

    public ListOfNumbers () {
        list = new ArrayList<Integer>(SIZE);
        for (int i = 0; i < SIZE; i++) {
            list.add(new Integer(i));
        }
    }

    public void writeList() {
    // The FileWriter constructor throws IOException, which must be caught.
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));

        for (int i = 0; i < SIZE; i++) {
            // The get(int) method throws IndexOutOfBoundsException, which must be caught.
            out.println("Value at: " + i + " = " + list.get(i));
        }
        out.close();
    }
}
```

There are two Exceptional points

- The <u>constructor</u> initializes an output stream on a file. If the file cannot be opened, the constructor throws an *IOException*.
- A call to the ArrayList class's <u>get</u> method, which throws an *IndexOutOfBoundsException* if the value of its argument is too small (less than 0) or too large.

If you try to compile, the compiler prints an error message about the exception thrown by the *FileWriter constructor*. However, it does not display an error message about the exception thrown by *get*.
The reason is that the exception thrown by the constructor, *IOException*, is a **checked exception**, and the one thrown by the *get* method, *IndexOutOfBoundsException*, is an **unchecked exception**.

The try Block

- The first step in constructing an exception handler
  - is to enclose the code that might throw an exception within a **try block**.

- In general, a **try block** looks like the following:

```
try {
    code
}
catch and finally blocks . . .
```

- To make an exception handler for a method
  - You can put each line of code that might throw an exception within its own try block and provide separate exception handlers for each.
  - Or, you can put all the code within a single try block and associate multiple handlers with it.
- If an exception occurs within the try block,
  - that exception is handled by an exception handler associated with it.
  - To associate an exception handler with a try block, you must put a **catch block** after it;

The catch Blocks

```
try {

} catch (ExceptionType name) {

} catch (ExceptionType name) {

}
```

- You associate exception handlers with a **try block**
  - by providing one or more **catch blocks** directly after the try block.
- Each catch block
  - is an exception handler that handles the type of exception indicated by its argument.
  - The argument type, ***ExceptionType***, declares the type of exception that the handler can handle and must be the name of a class that inherits from the *Throwable* class.
  - The handler can refer to the exception with ***name***.
- The catch block contains code
  - that is executed if and when the exception handler is invoked.
  - The runtime system invokes the exception handler when the handler is the first one in the call stack whose *ExceptionType* matches the type of the exception thrown.
- Exception handlers
  - can do more than just print error messages or halt the program.
  - They can do error recovery, prompt the user to make a decision, or propagate the error up to a higher-level handler using chained exceptions,

The finally Block

- The **finally block** always executes
  - when the **try block** exits.
  - This ensures that the finally block is executed even if an unexpected exception occurs.
- The runtime system
  - Always executes the statements within the finally block regardless of what happens within the try block.
  - So it's the perfect place to perform cleanup.

```
try {

} catch (ExceptionType name) {

} catch (ExceptionType name) {

}finally {

}
```

- Important:
  - The finally block is a key tool for preventing resource leaks.
  - When closing a file or otherwise recovering resources, place the code in a finally block to ensure that resource is always recovered.

```java
public void writeList() {
    PrintWriter out = null;

    try {
        System.out.println("Entering" + " try statement");

        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++) {
            out.println("Value at: " + i + " = " + list.get(i));
        }
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: "+  e.getMessage());

    } catch (IOException e) {
        System.err.println("Caught IOException: " +  e.getMessage());

    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        }else{
            System.out.println("PrintWriter not open");
        }
    }
}
```

Specifying the Exceptions Thrown by a Method

- It may be appropriated for code to catch exceptions that can occur within it.
- In other cases, however, it's better to let a method further up the call stack handle the exception.
  - For example, if you were providing the *ListOfNumbers* class as part of a package of classes. In this case, it's better to not catch the exception and to allow a method further up the call stack to handle it.
- If a method
  - doesn't catch the **checked exceptions** that can occur within it,
  - the method must specify that it can throw these exceptions.
- To specify that a method can throw exceptions,
  - add a **throws clause** to the method declaration.
  - The **throws clause** comprises the ***throws*** keyword followed by a comma-separated list of all the exceptions thrown by that method.
  - The clause goes after the method name and argument list and before the brace that defines the scope of the method;
- **Including unchecked exception in the throws clause is not mandatory.**

```
public void writeList() throws IOException {
    PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));

    for (int i = 0; i < SIZE; i++) {
        out.println("Value at: " + i + " = " + list.get(i));
    }
    out.close();
}
```

How to Throw Exceptions

- Before you can catch an exception,
  - some code somewhere must throw one.
  - Any code can throw an exception: your code, code from the packages that come with the Java platform, or the Java runtime environment.

- Regardless of what throws the exception,
  - it's always thrown with the **throw** statement.

- The Java platform
  - provides numerous exception classes.
  - All the classes are descendants of the ***Throwable*** class,
  - and all allow programs to differentiate among the various types of exceptions that can occur during the execution of a program.

- You can also create your own exception classes to represent problems that can occur within the classes you write.

How to Throw Exceptions

- **The throw Statement**
  - All methods use the **throw statement** to throw an exception.
  - The throw statement requires a single argument: a *throwable* object.
  - **Throwable objects** are instances of any subclass of the ***Throwable*** class.
  - Example: throw someThrowableObject;
- You need to remember that you can throw only objects that inherit from the java.lang.Throwable class.

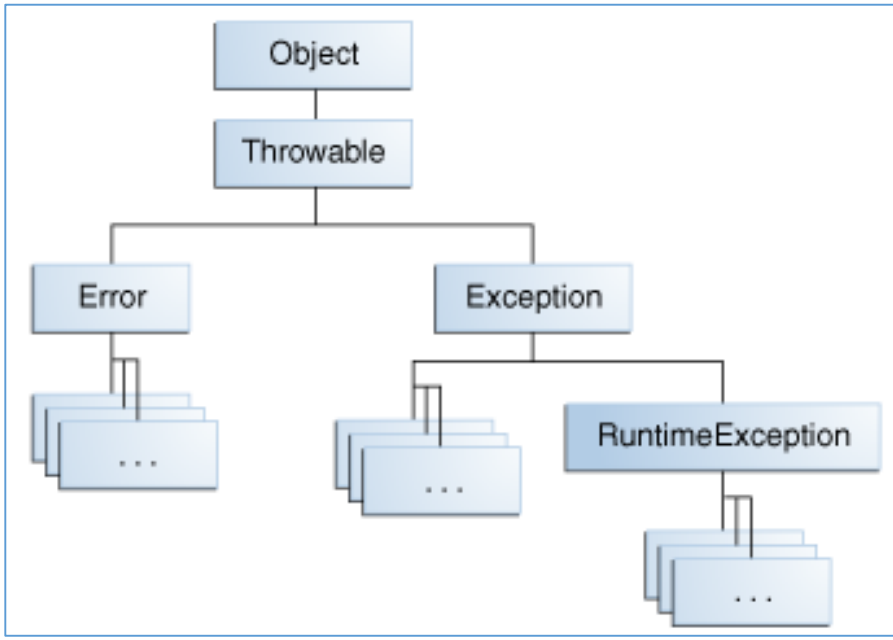
```
public Object pop() {
    Object obj;

    if (size == 0) throw new EmptyStackException();

    obj = objectAt(size - 1);
    setObjectAt(size - 1, null);
    size--;
    return obj;
}
```

Note that the declaration of the pop method does not contain a throws clause. *EmptyStackException* is not a checked exception, so pop is not required to state that it might occur. *EmptyStackException* is a class member of *java.util* package

How to Throw Exceptions



- **Throwable Class and Its Subclasses**
  - When a dynamic linking failure or other hard failure in the Java virtual machine occurs, the virtual machine throws an ***Error***.
  - The Java platform defines the many descendants of the ***Exception*** class. These descendants indicate various types of exceptions that can occur.
  - For example, *IllegalAccessException* signals that a particular method could not be found, and *NegativeArraySizeException* indicates that a program attempted to create an array with a negative size.
  - One Exception subclass, *RuntimeException*, is reserved for exceptions that indicate incorrect use of an API.
  - An example of a runtime exception is *NullPointerException*, which occurs when a method tries to access a member of an object through a null reference.

Chained Exceptions

- An application
  - often responds to an exception by throwing another exception.
  - In effect, the first exception causes the second exception.
  - It can be very helpful to know when one exception causes another.
  - Chained Exceptions help the programmer do this.
- The following are the methods and constructors in Throwable that support chained exceptions.
  - Throwable getCause()
  - Throwable initCause(Throwable)
  - Throwable(String, Throwable)
  - Throwable(Throwable)
- The Throwable argument to *initCause* and the Throwable constructors is the exception that caused the current exception.
- *getCause* returns the exception that caused the current exception, and *initCause* sets the current exception's cause.

```
try {

} catch (IOException e) {
   throw new SampleException("Other IOException", e);
}
```

Creating Exception Classes

- When faced with choosing the type of exception to throw,
  - you can either use one written by someone else — the Java platform provides a lot of exception classes you can use
  - or you can write one of your own.
- **Choosing a Superclass**
  - Any Exception subclass can be used as the parent class of your Exception class.
  - However, the parent class of your Exception class May better be *Exception*.

- Most applications you write
  - will throw objects that are Exceptions.
  - Errors are normally used for serious, hard errors in the system, such as those that prevent the JVM from running.

- Note:
  - For readable code, it's good practice to append the string **Exception** to the names of all classes that inherit (directly or indirectly) from the **Exception** class.

Unchecked Exceptions — The Controversy

- Why did the designers decide to force a method to specify all uncaught checked exceptions that can be thrown within its scope?
  - Any Exception that can be thrown by a method is part of the method's public programming interface.
  - Those who call a method must know about the exceptions that a method can throw
  - so that they can decide what to do about them.
  - These exceptions are as much a part of that method's programming interface as its parameters and return value.

- One case where it is common practice to throw a *RuntimeException*
  - is when the user calls a method incorrectly.
  - For example, a method can check if one of its arguments is incorrectly null.
  - If an argument is null, the method might throw a *NullPointerException*, which is an unchecked exception.

- If a client can reasonably be expected to recover from an exception, make it a **checked exception**.
- If a client cannot do anything to recover from the exception, make it an **unchecked exception**.

Advantages of Exceptions

- **Separating Error-Handling Code from "Regular" Code**
  - Exceptions enable you to write the main flow of your code and to deal with the exceptional cases elsewhere.

- **Propagating Errors Up the Call Stack**
  - A method can duck any exceptions thrown within it, thereby allowing a method farther up the call stack to catch it. Hence, only the methods that care about errors have to worry about detecting errors

- **Grouping and Differentiating Error Types**
  - A method can catch an exception based on its group or general type by specifying any of the exception's superclasses in the catch statement.
  - You could even set up an exception handler that handles any Exception with the handler here.

```
catch (Exception e) {
    ...
}
```

    - You may want to handle exceptions this way if all you want your program to do, for example, is print out an error message for the user and then exit.

```java
import java.io.*;
public class application {

    public static void main(String[] args){
        String userInput="";
        System.out.print("Insert Your Name: ");

        try{
            while(true){
                int x=System.in.read();
                userInput+=Character.toChars(x)[0];
                if(x==10)break;
            }
        }catch(IOException e){
            System.out.println(e.getMessage());
        }
        System.out.println("\nYour Name is: "+userInput);
        System.out.println("Other line");
    }
}
```

```java
import java.io.*;
public class application {

    public static void main(String[] args) throws IOException{
        String userInput="";
        System.out.print("Insert Your Name: ");


        while(true){
            int x=System.in.read();
            userInput+=Character.toChars(x)[0];
            if(x==10)break;
        }

        System.out.println("\nYour Name is: "+userInput);
        System.out.println("Other line");
    }
}
```

Bicycle Example

```java
public class BicycleExeption extends Exception {
    public BicycleExeption (){
        super();
    }
    public BicycleExeption (String message){
        super(message);
    }
}
```

```java
public class Bicycle {
    //...
    public Bicycle(int startCadence,int startSpeed,int startGear) throws BicycleExeption{
        if(startSpeed<0)throw new BicycleExeption("Negative Speed!");
        //...
    }
    //...
}
```

```java
import java.util.Scanner;

public class program {
    private static Scanner myScanner = new Scanner(System.in);

    public static void main(String[] args){
        Bicycle yourBike = null;
        System.out.println("What is your Bike's initial speed?");
        int x = myScanner.nextInt();

        try{
            yourBike = new Bicycle(0,x,0);
        }catch (BicycleExeption e){
            System.out.println("Error in making a bike");
            throw new RuntimeException("Program Stopped",e);
        }

        System.out.println("Your Bike ID:"+yourBike.getID()+" speed:"+yourBike.getSpeed());

    }
}
```

Exercise

- Use a similar approach of the previous example “Bicycle Example” to add appropriate **exceptions** to your ClassTV that you have written for Slide 7’s exercise.
  Handle the thrown exceptions in your main method.